

به نام خدا



دانشگاه تهران  
دانشکده مهندسی برق و کامپیوتر



## درس مدلسازی و درستی‌یابی صوری

گزارش تمرین برنامه‌نویسی ۱

نام و نام خانوادگی :  
فاطمه عبدلی  
هانیه گرگانی

شماره دانشجویی:  
۸۱۰۱۰۴۲۰۲  
۸۱۰۱۰۴۲۳۸

آبان ۱۴۰۴

## 1. INTRODUCTION AND GOALS

The First phase of this project established a basic communication model for three satellites and a ground station, employing a Time Division Multiple Access (TDMA) protocol with 8 slots (0-7) to manage inter-satellite (ISL) and satellite-to-ground communications to avoid conflict and congestion. Also message buffering was implemented due to asynchronous message passing. This phase enhances the model's functionalities and reliability through two major additions which are Energy Management and Formal Verification.

### 1.1 Energy Management

This part developed in order to simulate realistic power constraints and system behavior under low energy to prevent mission-critical failure and ensure autonomous recovery.

### 1.2 Formal Verification

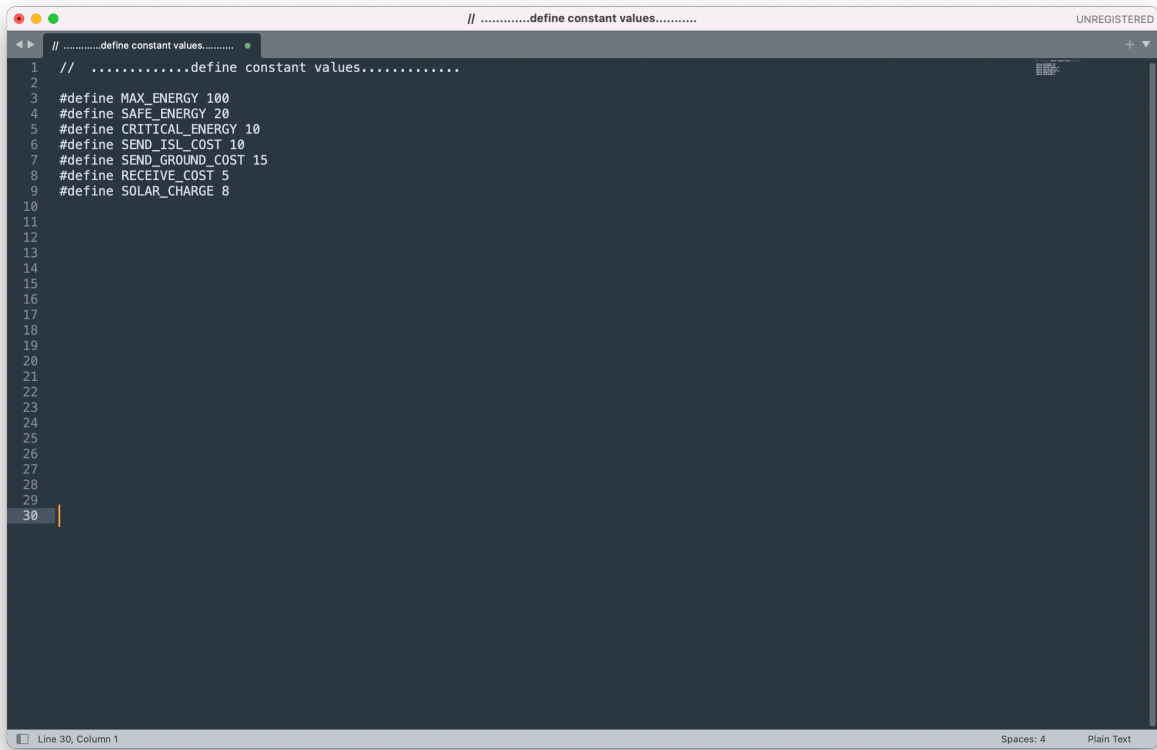
LTL properties and assertions prove system correctness. Checks for Safety- no ground conflict, no low-energy send, and no buffer overflow- and Liveness- guaranteed message delivery/drop, and periodic ground access- properties are implemented.

## 2. STRUCTURE AND CODE

The model is written in *Promela* (Process Meta Language) for the *Spin* Model Checker, and uses channels for communication and asynchronous message passing.

### 2.1 Constant values

This section defines the core parameters, particularly the energy management thresholds and costs that govern the satellite behavior. Constants shown in Figure1.1 clearly establish the power budget. Sending to the Ground (15



```
// .....define constant values.....  
1 // .....define constant values.....  
2  
3 #define MAX_ENERGY 100  
4 #define SAFE_ENERGY 20  
5 #define CRITICAL_ENERGY 10  
6 #define SEND_ISL_COST 10  
7 #define SEND_GROUND_COST 15  
8 #define RECEIVE_COST 5  
9 #define SOLAR_CHARGE 8  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30
```

The screenshot shows a code editor window with a dark background. The title bar at the top reads "UNREGISTERED". The code is in C preprocessor syntax, defining several constants. Line numbers 1 through 30 are visible on the left margin. The code defines MAX\_ENERGY as 100, SAFE\_ENERGY as 20, CRITICAL\_ENERGY as 10, SEND\_ISL\_COST as 10, SEND\_GROUND\_COST as 15, RECEIVE\_COST as 5, and SOLAR\_CHARGE as 8. The cursor is at line 30, column 1. The status bar at the bottom indicates "Line 30, Column 1", "Spaces: 4", and "Plain Text".

**Figure1.1- definition of constant values**

units) is the most expensive operation, promoting a conservative approach, while Solar charge (8 units) is the recharging amount. The Safe (20) and Critical (10) bounds dictate the satellite's ability to operate.

## **2.2 Timekeeper and Energy Recharge**

The timekeeper process guarantees the system's **Liveness** by ensuring energy levels trend upwards. Critically, as shown in Figure1.2 it checks the Safe Mode exit condition and resets the `safe_mode` flag, allowing the satellite to resume transmitting.

```
// .....timekeeper processes.....
1 // .....timekeeper processes.....
2
3
4 proctype timekeeper()
5 {
6     atomic {
7         if
8             :: time_signal ! current_slot ->
9             current_slot = (current_slot + 1) % N;
10
11         // Recharge satellite energy from solar panel each time slot
12         int i = 0;
13         do
14             :: i < satellite_num ->
15             if
16                 :: (energy_satellite[i] + SOLAR_CHARGE) <= MAX_ENERGY ->
17                 energy_satellite[i] = energy_satellite[i] + SOLAR_CHARGE;
18             :: else ->
19                 energy_satellite[i] = MAX_ENERGY;
20             fi;
21
22             if
23                 :: safe_mode[i] && energy_satellite[i] >= SAFE_ENERGY ->
24                 safe_mode[i] = false;
25                 printf("satellite(%d) exiting Safe Mode, energy: %d\n", i+1, energy_satellite[i]);
26             :: else -> skip
27             fi;
28             i = i + 1
29             :: else -> break
30         od
31     fi
32 }
33
34
35
36
37
38
39
40
41
42
43
Line 41, Column 1
```

Figure1.2- definition of timekeeper process

```
// .....Coordinator processes.....
1 // .....Coordinator processes.....
2
3
4 proctype coordinator()
5 {
6     if
7         :: time_signal ? slot ->
8         if
9             :: slot == 0 -> grant_ground[0] ! 1;
10            :: slot == 1 -> grant_ground[1] ! 1;
11            :: slot == 2 -> grant_ground[2] ! 1;
12            :: slot == 3 -> printf("Synchronization slot\n");
13            :: slot == 4 -> grant_isl[0] ! 12;
14            :: slot == 5 -> grant_isl[1] ! 23;
15            :: slot == 6 -> grant_isl[2] ! 13;
16            :: slot == 7 -> printf("Synchronization slot\n");
17        fi
18    fi
19 }
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
Line 35, Column 1
```

Figure1.3- definition of coordinator process



## 2.3 TDMA Coordinator

The coordinator process shown in Figure 1.3 manages the 8-slot TDMA schedule and grants access to the shared communication channels. In this phase we've fixed schedule to ensure **Liveness** periodic access property. Satellites 1, 2, and 3 get dedicated ground slots 0, 1, 2 and dedicated ISL slots 4, 5, 6. Slots 3 and 7 are reserved for synchronization as before.

## 2.4 Satellite

### 2.4.1 Receiving phase

In this section two key safety rules are enforced, shown in Figure 1.4:

- **No Buffer Overflow:** The `assert(...)` explicitly checks this **safety property**.
- **Safe Mode Entry:** If the satellite cannot afford the `RECEIVE_COST` which is 5 units, or if the cost pushes it below the **Critical Energy Bound**, it enters `safe_mode`.

### 2.4.2 Sending Phase

Here **TDMA protocol** and **Energy Management** rules are strictly enforced before a message can be transmitted. The process first checks if there is a message ready to be sent, and if is, it checks the satellite's energy level to be upper both safe and critical bounds. And if not it will skip the slot as shown in Figure 1.5.

```

1 // .....satellite processes.....
2
3
4 proctype satellite1()
5 {
6     MESSAGE buff(buffer_cap), temp_message_receive, temp_message_send;
7     tail_sat1 = 0;
8     head_sat1 = 0;
9     bool is_turn_send_ground = false;
10    int is_turn_send_isl12 = 0;
11    int is_turn_send_isl13 = 0;
12
13    // .....receiving phase.....
14
15    do
16    :: ISL[0] ? temp_message_receive ->
17        // Ensure buffer is not full before adding a new message
18        assert( (tail_sat1 + 1) % buffer_cap != head_sat1 );
19        buff[tail_sat1].message_type = temp_message_receive.message_type;
20        buff[tail_sat1].sender_satellite_id = temp_message_receive.sender_satellite_id;
21        buff[tail_sat1].receiver_satellite_id = temp_message_receive.receiver_satellite_id;
22        buff[tail_sat1].payload = temp_message_receive.payload;
23        printf("satellite(1) buffered message {type: %d, sender : %d, receiver: %d, payload: %d}\n",
24            buff[tail_sat1].message_type, buff[tail_sat1].sender_satellite_id,
25            buff[tail_sat1].receiver_satellite_id, buff[tail_sat1].payload);
26        tail_sat1 = (tail_sat1 + 1) % buffer_cap;
27
28        /* Energy consumption for receiving data */
29        if
30        :: energy_satellite[0] >= RECEIVE_COST ->
31            energy_satellite[0] = energy_satellite[0] - RECEIVE_COST;
32            printf("satellite(1) received a message, energy now: %d\n", energy_satellite[0]);
33        :: else ->
34            safe_mode[0] = true;
35            printf("satellite(1) entering Safe Mode due to low energy on receive (%d)\n", energy_satellite[0]);
36        fi;
37    :: skip -> goto sendingPhase1
38    od
39
40    // .....sending phase.....
41
42    sendingPhase1:
43    }
146 }

```

Figure1.4- definition of satellite process (receiving phase)

```

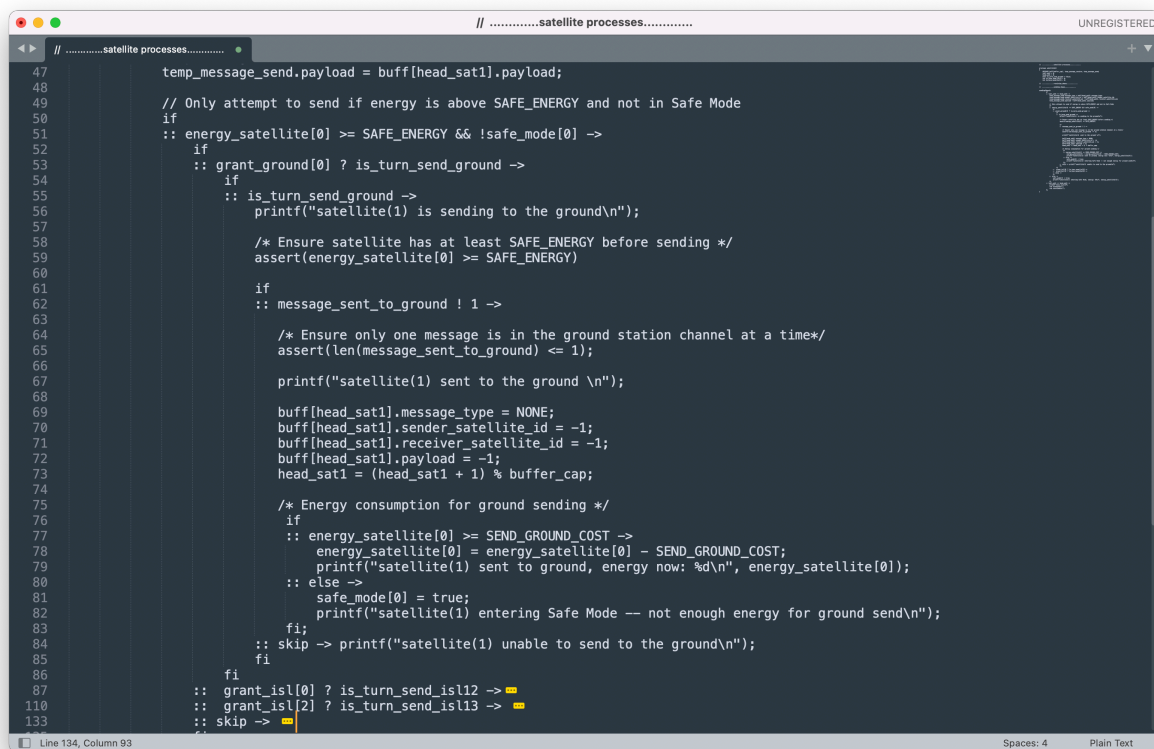
1 // .....satellite processes.....
2
3
4 proctype satellite1()
5 {
6     MESSAGE buff(buffer_cap), temp_message_receive, temp_message_send;
7     tail_sat1 = 0;
8     head_sat1 = 0;
9     bool is_turn_send_ground = false;
10    int is_turn_send_isl12 = 0;
11    int is_turn_send_isl13 = 0;
12
13    // .....receiving phase.....
14
15    // .....sending phase.....
16
17    sendingPhase1:
18    if
19    :: tail_sat1 != head_sat1 ->
20        temp_message_send.message_type = buff[head_sat1].message_type;
21        temp_message_send.sender_satellite_id = buff[head_sat1].sender_satellite_id;
22        temp_message_send.receiver_satellite_id = buff[head_sat1].receiver_satellite_id;
23        temp_message_send.payload = buff[head_sat1].payload;
24
25        // Only attempt to send if energy is above SAFE_ENERGY and not in Safe Mode
26        if
27        :: energy_satellite[0] >= SAFE_ENERGY && !safe_mode[0] ->
28            if
29            :: grant_ground[0] ? is_turn_send_ground ->
30            :: grant_isl[0] ? is_turn_send_isl12 ->
31            :: grant_isl[2] ? is_turn_send_isl13 ->
32            :: skip ->
33            fi
34            temp_message_send.receiver_satellite_id = 0;
35            temp_message_send.payload = 0;
36            printf("satellite(1) sending message, energy: %d\n", energy_satellite[0]);
37            head_sat1 = (head_sat1 + 1) % buffer_cap;
38            tail_sat1 = (tail_sat1 + 1) % buffer_cap;
39            if
40            :: grant_ground[0] ? is_turn_send_ground ->
41            :: grant_isl[0] ? is_turn_send_isl12 ->
42            :: grant_isl[2] ? is_turn_send_isl13 ->
43            :: skip ->
44            fi
45            safe_mode[0] = false;
46            printf("satellite(1) leaving Safe Mode, energy: %d\n", energy_satellite[0]);
47        fi
48    :: tail_sat1 == head_sat1 ->
49        printf("skip slot\n");
50        run timekeeper();
51        run coordinator();
52    fi
53
54    }
146 }

```

Figure1.5- definition of satellite process (sending phase)

### 2.4.2.1 Send to the Ground

As shown in Figure1.6, if the satellite can send to the ground due to its turn and there is a message to be sent, the assert (energy\_satellite[0] >= SAFE\_ENERGY) reinforces no low-energy **safety property**, and also the assert (len(message\_sent\_to\_ground) <= 1) prevents Ground conflict **safety property**. Afterward, a high-cost ground transmission is processed, and if the remainder is low, the satellite enters Safe Mode.

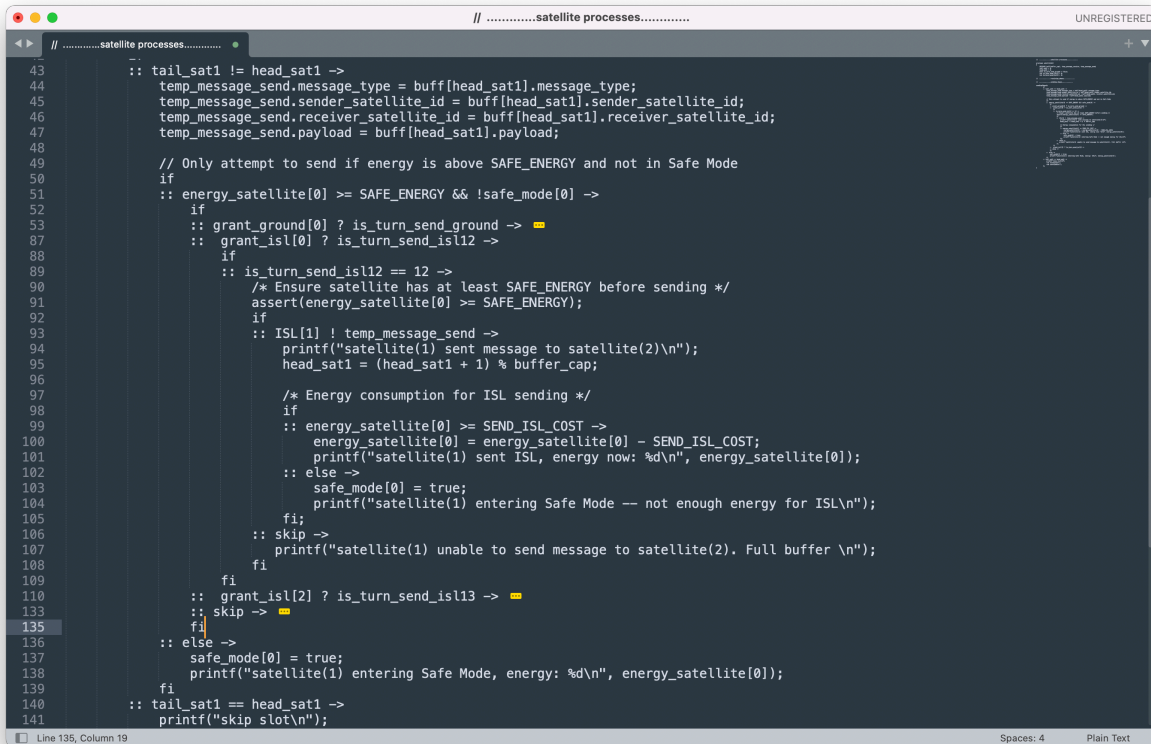


```
// .....satellite processes.....
47 temp_message_send.payload = buff[head_sat1].payload;
48
49 // Only attempt to send if energy is above SAFE_ENERGY and not in Safe Mode
50 if
51 :: energy_satellite[0] >= SAFE_ENERGY && !safe_mode[0] ->
52 if
53 :: grant_ground[0] ? is_turn_send_ground ->
54 if
55 :: is_turn_send_ground ->
56 printf("satellite(1) is sending to the ground\n");
57
58 /* Ensure satellite has at least SAFE_ENERGY before sending */
59 assert(energy_satellite[0] >= SAFE_ENERGY)
60
61 if
62 :: message_sent_to_ground ! 1 ->
63
64 /* Ensure only one message is in the ground station channel at a time*/
65 assert(len(message_sent_to_ground) <= 1);
66
67 printf("satellite(1) sent to the ground \n");
68
69 buff[head_sat1].message_type = NONE;
70 buff[head_sat1].sender_satellite_id = -1;
71 buff[head_sat1].receiver_satellite_id = -1;
72 buff[head_sat1].payload = -1;
73 head_sat1 = (head_sat1 + 1) % buffer_cap;
74
75 /* Energy consumption for ground sending */
76 if
77 :: energy_satellite[0] >= SEND_GROUND_COST ->
78 energy_satellite[0] = energy_satellite[0] - SEND_GROUND_COST;
79 printf("satellite(1) sent to ground, energy now: %d\n", energy_satellite[0]);
80 :: else ->
81 safe_mode[0] = true;
82 printf("satellite(1) entering Safe Mode -- not enough energy for ground send\n");
83 fi;
84 :: skip -> printf("satellite(1) unable to send to the ground\n");
85 fi
86 fi
87
88 :: grant_isl[0] ? is_turn_send_isl12 ->
89 :: grant_isl[2] ? is_turn_send_isl13 ->
90 :: skip ->
```

Figure1.6- definition of satellite process (sending phase - Send to the Ground)

### 2.4.2.2 Send to other satellites

At the very beginning of this part, the no low-energy **safety property** is checked by `assert(energy_satellite[i] >= SAFE_ENERGY)` and then the consumption logic is applied, ensuring all transmission activities are energy-aware and guarded against low-power operation. Each satellite (i) can send to two other satellites (j) and (k) in the orbit and the sending parts are logically the same but divided by two guards in the If Statement, shown in Figure1.7 and Figure1.8.



```
// .....satellite processes.....
43  :: tail_sat1 != head_sat1 ->
44    temp_message_send.message_type = buff[head_sat1].message_type;
45    temp_message_send.sender_satellite_id = buff[head_sat1].sender_satellite_id;
46    temp_message_send.receiver_satellite_id = buff[head_sat1].receiver_satellite_id;
47    temp_message_send.payload = buff[head_sat1].payload;
48
49    // Only attempt to send if energy is above SAFE_ENERGY and not in Safe Mode
50    if
51    :: energy_satellite[0] >= SAFE_ENERGY && !safe_mode[0] ->
52      if
53      :: grant_ground[0] ? is_turn_send_ground ->
54      :: grant_isl[0] ? is_turn_send_isl12 ->
55        if
56        :: is_turn_send_isl12 == 12 ->
57          /* Ensure satellite has at least SAFE_ENERGY before sending */
58          assert(energy_satellite[0] >= SAFE_ENERGY);
59          if
60          :: ISL[1] ! temp_message_send ->
61            printf("satellite(1) sent message to satellite(2)\n");
62            head_sat1 = (head_sat1 + 1) % buffer_cap;
63
64            /* Energy consumption for ISL sending */
65            if
66            :: energy_satellite[0] >= SEND_ISL_COST ->
67              energy_satellite[0] = energy_satellite[0] - SEND_ISL_COST;
68              printf("satellite(1) sent ISL, energy now: %d\n", energy_satellite[0]);
69            :: else ->
69              safe_mode[0] = true;
70              printf("satellite(1) entering Safe Mode -- not enough energy for ISL\n");
71            fi;
72          :: skip ->
73            printf("satellite(1) unable to send message to satellite(2). Full buffer \n");
74          fi
75        fi
76      fi
77      :: grant_isl[2] ? is_turn_send_isl13 ->
78      :: skip ->
79    fi
80  :: else ->
81    safe_mode[0] = true;
82    printf("satellite(1) entering Safe Mode, energy: %d\n", energy_satellite[0]);
83  fi
84  :: tail_sat1 == head_sat1 ->
85    printf("skip slot\n");
86  fi
```

Figure1.7- definition of satellite process (sending phase - Satellite (i) send to the satellite (j))

```

// .....satellite processes.....
43  --
44  :: tail_sat1 != head_sat1 ->
45    temp_message_send.message_type = buff[head_sat1].message_type;
46    temp_message_send.sender_satellite_id = buff[head_sat1].sender_satellite_id;
47    temp_message_send.receiver_satellite_id = buff[head_sat1].receiver_satellite_id;
48    temp_message_send.payload = buff[head_sat1].payload;
49
50    // Only attempt to send if energy is above SAFE_ENERGY and not in Safe Mode
51    if
52    :: energy_satellite[0] >= SAFE_ENERGY && !safe_mode[0] ->
53      if
54      :: grant_ground[0] ? is_turn_send_ground -> ==
55      :: grant_isl[0] ? is_turn_send_isl12 -> ==
56      :: grant_isl[2] ? is_turn_send_isl13 ->
57      if
58      :: is_turn_send_isl13 == 13 ->
59        /* Ensure satellite has at least SAFE_ENERGY before sending */
60        assert(energy_satellite[0] >= SAFE_ENERGY);
61        if
62        :: ISL[2] ! temp_message_send ->
63          printf("satellite(1) sent message to satellite(3) \n");
64          head_sat1 = (head_sat1 + 1) % buffer_cap;
65
66          /* Energy consumption for ISL sending */
67          if
68          :: energy_satellite[0] >= SEND_ISL_COST ->
69            energy_satellite[0] = energy_satellite[0] - SEND_ISL_COST;
70            printf("satellite(1) sent ISL, energy now: %d\n", energy_satellite[0]);
71            :: else ->
72              safe_mode[0] = true;
73              printf("satellite(1) entering Safe Mode -- not enough energy for ISL\n");
74              fi;
75            :: skip ->
76            printf("satellite(1) unable to send message to satellite(3). Full buffer \n");
77            fi
78          fi
79          :: skip -> ==
80          fi
81          :: else ->
82            safe_mode[0] = true;
83            printf("satellite(1) entering Safe Mode, energy: %d\n", energy_satellite[0]);
84            fi
85          :: tail_sat1 == head_sat1 ->
86            printf("skip slot\n");
87

```

**Figure1.8- definition of satellite process (sending phase - Satellite (i) send to the satellite (k))**

## 2.5 Ground Processes

The groundStation process remained the same as what was in the Version1 as shown in Figure1.9; receives messages and confirms successful transmission and records the number of messages per satellite.



```

// .....groundStation processes.....
// .....groundStation processes.....
proctype groundStation() {
    int temp_message;

    if
    :: message_sent_to_ground ? temp_message ->
    if
    :: temp_message == 1 ->
        message_num_per_satellite[0] = message_num_per_satellite[0] + 1;
        printf("satellite(1) sent %d message(s) to the ground! \n", message_num_per_satellite[0]);
    :: temp_message == 2 ->
        message_num_per_satellite[1] = message_num_per_satellite[1] + 1;
        printf("satellite(2) sent %d message(s) to the ground! \n", message_num_per_satellite[1]);
    :: temp_message == 3 ->
        message_num_per_satellite[2] = message_num_per_satellite[2] + 1;
        printf("satellite(3) sent %d message(s) to the ground! \n", message_num_per_satellite[2]);
    :: else -> printf("No buffered message from satellite(1) sent to the ground at the moment! \n");
    fi
    :: skip -> printf("ground receiving buffer blocked! \n");
    fi
}

```

Line 46, Column 1      Spaces: 4      Plain Text

**Figure1.9- definition of groundStation process**

```

// .....define LTL conditions.....
// .....define LTL conditions.....
/* Ensure that whenever there are messages in the buffer (tail != head),
eventually all messages are processed (tail catches up to head)*/

// Always: if satellite 1's buffer is not empty, it will eventually be processed and become empty
ltl sat1_processed { [] (tail_sat1 != head_sat1 -> <= (tail_sat1 == head_sat1)) }

// Always: if satellite 2's buffer is not empty, it will eventually be processed and become empty
ltl sat2_processed { [] (tail_sat2 != head_sat2 -> <= (tail_sat2 == head_sat2)) }

// Always: if satellite 3's buffer is not empty, it will eventually be processed and become empty
ltl sat3_processed { [] (tail_sat3 != head_sat3 -> <= (tail_sat3 == head_sat3)) }

/* Ensure that the ground station grant signal for each satellite is given infinitely often,
i.e., each satellite periodically gets access to send data to the ground*/

// Always: satellite 1 will eventually receive permission to send to the ground (periodic ground access)
ltl periodic_access_0 { [] (<= grant_ground[0]) }

// Always: satellite 2 will eventually receive permission to send to the ground (periodic ground access)
ltl periodic_access_1 { [] (<= grant_ground[1]) }

// Always: satellite 3 will eventually receive permission to send to the ground (periodic ground access)
ltl periodic_access_2 { [] (<= grant_ground[2]) }

```

Line 45, Column 1      Spaces: 4      Plain Text

**Figure1.10- definition of LTL properties**

## 2.6 LTL Properties

Two series of LTL properties are defined as shown in Figure 1.10: **Guaranteed Message Processing** and **Periodic Access**. These properties are crucial for formal verification, ensuring the satellite system does not suffer from permanent stalls or starvation.

### 2.6.1 Guaranteed Message Processing

These LTL properties ensure that messages placed in a satellite's buffer are not kept indefinitely and no data starvation happens; they must *eventually* be sent or dropped. If Satellite (i)'s buffer is **not empty**, it must **eventually** become **empty**.

### 2.6.2 Periodic Ground Access

These LTL properties ensure that the TDMA scheduling mechanism provides every satellite with an opportunity to communicate with the ground station infinitely often, since it is cyclical. It is **eventually** true that Satellite (i) receives the **ground grant** channel.

## 3. HOW TO RUN

### 3.1 Prerequisites

Spin Model Checker: The core tool required to run *Promela* code.

### 3.2 Execution

To see the flow of messages, energy levels, and Safe Mode entries/exits, clone repository:

[Git clone https://github.com/HaniehGRN/inter-satellite-communication](https://github.com/HaniehGRN/inter-satellite-communication)

And run a non-deterministic simulation in *jspin*!

### 4. Analysis and Results

The model correctly enforces the defined safety requirements which guarantee that specific, undesirable states are unreachable:

#### 4.1 Safety Properties

##### 4.1.1 No Conflict in Ground Channel

- **Assertion:** `assert(len(message_sent_to_ground) <= 1);`
- **Result:** This is checked immediately after a satellite attempts to send to the ground, ensuring that no more than one message ever enters the ground channel at a time.

##### 4.1.2 No Sending Below Safe Energy

- **Assertion:** `assert(energy_satellite[i] >= SAFE_ENERGY);`
- **Result:** This check is executed before a send operation confirming the **Guard Logic** (`energy_satellite[i] >= SAFE_ENERGY && !safe_mode[i]`) is effective.

##### 4.1.3 No Buffer Overflow

- **Assertion:** `assert( (tail_sat(i) + 1) % buffer_cap != head_sat(i) );`
- **Result:** This circular buffer assertion prevents data loss due to attempting to write to an already full buffer.

#### 4.2 Liveness Properties



### 4.2.1 Guaranteed Message Delivery/Drop

- **LTL:**  $\square(\text{tail\_sat}(i) \neq \text{head\_sat}(i) \rightarrow \Diamond (\text{tail\_sat}(i) == \text{head\_sat}(i)))$
- **Result:** The TDMA slot guarantees sending opportunities, and the Safe Mode recovery ensures that even if a satellite runs out of energy, it will eventually recharge, exit Safe Mode, and process its buffer. This guarantees the buffer will not hold data indefinitely.

### 4.2.2 Periodic Ground Communication

- **LTL:**  $\square(\Diamond \text{grant\_ground}[i])$
- **Result:** The cyclical TDMA schedule - slots 0, 1, and 2 are dedicated to the three satellites - ensures that every satellite receives the ground grant infinitely often.

## 4.3 Energy Management

state transitions are modeled based on battery level, which has three stages: Normal Operation, between MAX\_ENERGY and SAFE\_ENERGY. Safe Mode, lower than SAFE\_ENERGY and CRITICAL\_ENERGY. And Critical lower than CRITICAL\_ENERGY. Each satellite gains energy at each slot, therefore, it can operate correctly.